Object Discovery from Oscillatory Timing in a Spiking Network

Anonymous submission

Abstract

We investigate whether the relative timing of oscillatory activity can guide object discovery in static images. Our spiking network maps image patches to coupled oscillators whose phase gates a spiking core, then groups patches by the temporal similarity of the emitted traces together with image-derived connectivity. The network trains without ground-truth masks and requires no prespecified object count at inference. Under a shared patch-level evaluation protocol, the model achieves a foreground adjusted Rand index of 0.841 on CLEVR, compared with 0.633 for the strongest evaluated baseline on that metric. Trained only on CLEVR, it also outperforms the evaluated baselines on Tetrominoes and Multi-dSprites without additional training. Ablations locate the grouping signal in cross-patch temporal alignment: destroying it collapses the partition, while substituting the oscillatory gate for the spike traces changes no mask.

1. Introduction

Object-centric models such as Slot Attention bind distributed image features by assigning them to shared object representations (Locatello et al. 2020). Oscillatory models such as AKOrN add synchronization to feature processing and obtain masks by clustering the resulting representations (Miyato et al. 2025). Both leave open a complementary question: can the timing of neural activity itself tell us which regions belong to the same object?

Neurons signal through brief electrical pulses, and both how often and when they fire can matter: cortical neurons reproduce spike timings under the same fluctuating input (Mainen and Sejnowski 1995). Brain rhythms make firing more likely at some points of a cycle than others, so when two populations synchronize their windows align (Fries 2005); in cat visual cortex the strength of synchrony between sites depended on how their local stimuli formed a larger pattern (Gray et al. 1989). S^2-Net (Dan and Wu 2026), which our core follows, uses coupled oscillators to regulate spiking.

We hypothesize that when oscillators representing patches of the same object synchronize, their downstream neurons produce similar spike patterns.

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

2. Method

Figure 1 shows the signal flow. A convolutional encoder turns the image into patch activations, which define a patch connection graph and an input phase for every oscillator. The oscillators evolve as a Kuramoto system on that graph, and their phase gates a dendritic and membrane cascade that emits spikes. The readout groups patches by the temporal similarity of those traces and merges the groups along the graph, one mask per discovered object. The oscillator-gated core follows S^2-Net (Dan and Wu 2026).

2.1 From Images to Neural Activations

2.1.1 Patch activation We encode an RGB image with eight convolutional filters, normalize and bound the feature maps, and pool them into a patch activation tensor $\gamma \in \mathbb{R}^{64\times64\times8}$. Each patch i has an eight-dimensional activation vector $\gamma_i \in \mathbb{R}^8$. The patch indices, i and j, identify image patches; the component index, a, identifies an oscillator component; and the time index, t, identifies a simulation step.

2.1.2 Patch connection graph In its neuroimaging setting, S^2-Net (Dan and Wu 2026) uses structural connectivity to set how strongly brain regions interact. We apply the same principle to images: a weighted graph controls how strongly patch oscillators influence one another, with edge weights based on feature similarity and geodesic distance.

$$S_{ij} = \frac{\boldsymbol{z}_i^\top \boldsymbol{z}_j}{\tau} - r\, d(i,j). \tag{1}$$

The connection score, S_{ij}, compares the normalized patch embeddings, $\boldsymbol{z}_i$ and $\boldsymbol{z}_j$, using the temperature, τ, and penalizes the geodesic distance, $d(i,j)$, using the spatial cost weight, r. τ and r are derived from trainable scalars.

We normalize the strongest connections and make their weights reciprocal.

$$\begin{aligned} B_{ij} &= G\ \mathrm{softmax}_{j\in\mathcal{N}_i}(S_{ij}), \\ A_{ij} &= \frac{B_{ij} + B_{ji}}{2}. \end{aligned} \tag{2}$$

For each patch i, the neighbor set, $\mathcal{N}_i$, contains the 512 patches with the highest connection scores, S_{ij}. The connection gain, G, is derived from a trainable scalar. The normalized connection weight, B_{ij}, is zero for unselected neighbors.

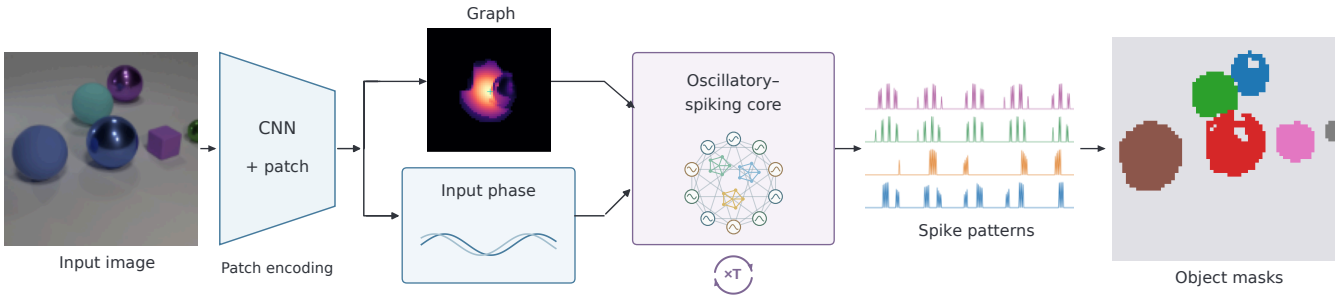


Figure 1: Model overview. CNN patch features define a patch graph and input phases. The oscillatory spiking core runs for T steps, and the readout combines temporal spike similarity with graph connectivity to group patches into object masks.

The symmetric connection weight, A_{ij}, defines the patch graph.

2.1.3 Input phase We project each patch activation with the trainable weight W_{drive}, standardize the result, and scale it with the trainable gain W_ϕ. A scaled tanh bounds the sensory drive, $\boldsymbol{\phi}_i$, to $[-\pi, \pi]$. We use the bounded drive as the initial oscillator phase (Eq. (A5)).

2.2 Recurrent Core

2.2.1 Kuramoto layer The Kuramoto layer coordinates oscillator phases across patches through intrinsic drift, patch coupling, and sensory drive. Each patch has a phase vector $\boldsymbol{\theta}_i(t) \in \mathbb{R}^4$, with one entry per oscillator.

$$\begin{aligned}\boldsymbol{\theta}_i(t+1) = \boldsymbol{\theta}_i(t) + \Delta t\Big(&\boldsymbol{\omega}_i + f\boldsymbol{\phi}_i \\ &+ \boldsymbol{\kappa}_i \odot \sin(\boldsymbol{\phi}_i - \boldsymbol{\theta}_i(t)) \\ &+ \frac{K}{N}\sum_j \bar{A}_{ij}\sin\big(\boldsymbol{\theta}_j(t) - \boldsymbol{\theta}_i(t) - \alpha_{ij}\big)\Big).\end{aligned} \tag{3}$$

The **natural frequency**, $\boldsymbol{\omega}_i$, gives each oscillator its intrinsic rate of phase change. The **coupling term**, the sum over j, adjusts the phase according to other patches, using the corrected connection weight, $\bar{A}_{ij}$, and the phase lag, α_{ij}. The **drive term** uses the sensory drive, $\boldsymbol{\phi}_i$, to adjust frequency through f and to pull the phase toward the input when the drive strength, $\boldsymbol{\kappa}_i$, is positive. The coupling scale, K, is normalized by the patch count, N. The time step, Δt, scales the update, and $\odot$ denotes elementwise multiplication. The parameters $\boldsymbol{\omega}_i$, f, and $\boldsymbol{\kappa}_i$ are trainable.

2.2.2 Sinusoidal gating The sinusoidal gating layer modulates the signal sent to the dendrites using a delayed phase. Each patch has one scalar gate $g_i(t) \in [0, 1]$, shared by its four oscillator components.

$$g_i(t) = \frac{1 + \sin(\bar{\theta}_i(\max(0, t-d)))}{2}. \tag{4}$$

$$\boldsymbol{x}_i(t) = g_i(t)\sin(\boldsymbol{\theta}_i(t)). \tag{5}$$

The gate $g_i(t)$ uses the mean phase of the four oscillators (Eq. (A8)) with a phase delay of d steps to modulate the dendritic input $\boldsymbol{x}_i(t)$.

2.2.3 Dendritic layer The dendritic layer integrates the gated input and spike feedback through four branches, whose states are summed to drive the membrane.

$$\boldsymbol{v}_{ia}(t) = U\begin{pmatrix} x_{ia}(t) \\ s_{ia}(t-1)\end{pmatrix} + \boldsymbol{b}. \tag{6}$$

The branch input, $\boldsymbol{v}_{ia}(t)$, combines the gated feature, $x_{ia}(t)$, and the previous emitted spike, $s_{ia}(t-1)$. The previous spike provides top-down feedback from the spiking output to the dendritic branches. The projection weight, $U \in \mathbb{R}^{4\times 2}$, and the bias, $\boldsymbol{b} \in \mathbb{R}^4$, are trainable.

The provisional state $\widehat{\boldsymbol{h}}_{ia}(t)$ balances the previous state and current input using $\boldsymbol{\beta}_i$ (Eq. (A9)).

$$\begin{aligned}\boldsymbol{h}_{ia}(t) &= \boldsymbol{h}_{ia}(t-1) + \big(1 - \eta_a(1 - g_i(t))\big) \\ &\quad \cdot \big(\widehat{\boldsymbol{h}}_{ia}(t) - \boldsymbol{h}_{ia}(t-1)\big), \\ H_{ia}(t) &= \sum_{b=1}^{4} h_{iab}(t).\end{aligned} \tag{7}$$

The gate and trainable weight η_a scale the state change. Summing over branches b yields the membrane input $H_{ia}(t)$.

2.2.4 Membrane layer The membrane layer accumulates dendritic input and generates spikes when its voltage crosses a threshold. The provisional voltage $\widehat{V}_{ia}(t)$ combines the previous voltage and current input using λ_i, with a reset from the previous spike (Eq. (A10)).

$$\begin{aligned}V_{ia}(t) &= V_{ia}(t-1) + \big(1 - \xi_a(1 - g_i(t))\big) \\ &\quad \cdot \big(\widehat{V}_{ia}(t) - V_{ia}(t-1)\big).\end{aligned} \tag{8}$$

$$\begin{aligned}e_{ia}(t) &= \text{ind}(V_{ia}(t) > \vartheta_a), \\ s_{ia}(t) &= g_i(t)e_{ia}(t).\end{aligned} \tag{9}$$

The gate and trainable weight ξ_a scale the voltage change. A threshold ϑ_a derived from a trainable parameter defines the binary event $e_{ia}(t)$; gating that event gives the emitted spike $s_{ia}(t)$.

2.3 Spike Grouping and Mask Generation

For each oscillator component, we mean-center and L2-normalize the spike trace from the last 512 simulation time steps. For patches i and j, the correlation ρ_{ija} is higher when

their spike activity tends to be above or below its respective average at the same time (Eq. (A12)). We compute the patch affinity, Q_{ij}, by multiplying the four correlations after clipping negative values to zero. A high affinity thus requires agreement across all four components: any nonpositive correlation makes Q_{ij} zero.

We form connected components by linking patch pairs with $Q_{ij} \geq 0.5$ and discard components smaller than 8 patches, a floor selected on CLEVR validation. Every patch is a foreground candidate; no firing-rate test is applied. As a heuristic, we treat the largest remaining component as background, and the components that survive form the foreground support.

We start the retained foreground patches as singleton groups and merge spatially adjacent groups using

$$E_{ij} = \left(1 + \frac{N\bar{A}_{ij}}{G}\right) \log \frac{Q'_{ij}}{1 - Q'_{ij}}, \qquad \Delta(\mathcal{C}, \mathcal{D}) = \sum_{i \in \mathcal{C}} \sum_{j \in \mathcal{D}} E_{ij}. \tag{10}$$

The stabilized affinity, Q'_{ij}, determines the sign of the merge evidence, E_{ij}. The merge score, $\Delta(\mathcal{C}, \mathcal{D})$, sums the evidence between the candidate patch groups, $\mathcal{C}$ and $\mathcal{D}$. Groups are merged while their score is positive. Each final group produces an object mask.

3. Experiments

3.1 Protocol

All models are evaluated by one function on one representation. Ground truth is quantised to a 64 × 64 patch grid and scored there; a prediction is never upsampled to pixels to be compared. A prediction coarser than the grid is lifted by an exact repeat, and one finer than the grid is reduced by patch majority vote, so every method is scored on the same object. The ceiling is therefore 1.0 by construction, and these numbers are not directly comparable with pixel-protocol figures reported elsewhere.

We report three metrics and never combine them: foreground adjusted Rand index (fg-ari), foreground IoU (fg-iou) and matched-object IoU (obj-iou). Background is label 0 on both sides, and the first two differ in which patches they see. fg-ari restricts both maps to the patches whose *ground truth* is foreground, so predicting an object on a true background patch is removed before scoring and is invisible to it. fg-iou merges all non-background identities per side and compares the two sets over their union, so false foreground enters the denominator; it is the metric that charges the error fg-ari cannot see. Appendix E gives exact definitions.

Matched exposure. Every model is trained on the same 10,000 CLEVR images for 10 epochs; for our model that is 6,250 updates at batch 16, i.e. 100,000 image exposures. Where a baseline's published recipe differs, only its epoch count is changed; architecture and optimiser are left at the authors' values. Tetrominoes and Multi-dSprites are never trained on: for every row in Table 1 those two columns are zero-shot transfer from the CLEVR-trained model. Training on the target and tuning on the target are separate conditions, and they do not hold equally for every row — Appendix E states which split fixed each model's free choices.

Readouts that require a cluster count. Several baselines obtain masks by clustering a continuous embedding, and the reference implementations of CAE, CAE++, CtCAE and SynCx set the number of clusters from the per-image ground-truth object count, $k_i = |\operatorname{unique}(y_i)|$, which makes their published figures oracle-assisted. We removed this: k becomes a per-dataset constant and the background mapping a rule, both fixed on *each target dataset's own* validation split — a weaker transfer condition than ours. No choice of any model is made on the reported split.

Minimum region size. Our readout emits many single-patch regions — 22.8% of predicted regions on CLEVR validation, 27.0% on Tetrominoes, 56.2% on Multi-dSprites — which appear when the greedy merge splits a scattered synchronous component, and which inflate the object count without materially changing the partition. We discard any predicted region smaller than a fraction f of that image's predicted foreground. The rule is scale-free and reads no ground truth; f = 0.01 is fixed on CLEVR validation and applied unchanged to both transfer datasets and to every baseline.

3.2 Generalisation across datasets

Under the matched training budget, our model scored higher on all nine dataset–metric combinations, at a single training seed. The reference row takes the best baseline in each cell separately, so it is a composite of three architectures that no single baseline attains. Which baseline is second depends on the dataset — AKOrN (conv) on Tetrominoes, AKOrN (attn)+up-tiling on Multi-dSprites, ItrSA on CLEVR matched-object IoU. The smallest margin is Tetrominoes foreground IoU, +0.0410; every other margin lies between 0.128 and 0.326, and we flag that cell rather than quoting only the aggregate.

What carries the grouping? Three measurements on the first 40 CLEVR reserved images, on the same rollouts, answer the hypothesis of Section 1 in sequence.

The pattern. Grouping patch pairs by ground truth — used only to group, never to set a threshold or a count — the affinity Q averages 0.891 ± 0.111 within one object, 0.047 ± 0.048 across objects and 0.099 ± 0.106 between object and background, a nineteenfold separation.

Is the alignment necessary? We roll each patch's four component traces by its own offset over the readout window, preserving every patch's rate and waveform and destroying only the alignment between patches. The readout's rules are unchanged, but every quantity they compute — affinity, background heuristic, merge — is recomputed from the shifted traces, so nothing is carried over from the control. FG-ARI falls from 0.865 to 0.074 and matched-object IoU from 0.693 to 0.033, while the mean predicted regions rise from 7.8 to 562. Foreground IoU falls only from 0.803 to 0.736, because it compares only the *union* of predicted non-background patches against the true foreground, and shattering one region into many leaves that union nearly unchanged.

Which signal carries it? With $s_{ia} = g_i e_{ia}$, the firing indicator e equals 1 in 99.99% of settled steps, so s follows

Model	CLEVR (in-domain)			Tetrominoes (zero-shot)			Multi-dSprites (zero-shot)		
	ari	f-iou	o-iou	ari	f-iou	o-iou	ari	f-iou	o-iou
Ours	.8408	.7528	.6387	.8568	.7950	.8170	.6966	.8190	.7625
AKOrN (attn) + up-tiling	.6326	.3468	.3636	.5484	.7230	.4167	.5505	.6539	.6348
AKOrN (conv)	.6046	.4219	.2769	.7202	.7540	.5913	.4055	.6834	.5434
AKOrN (attn)	.6027	.3327	.3750	.5676	.7383	.4687	.4343	.6019	.5751
ItrSA	.5794	.2787	.3853	.6319	.7137	.5687	.4817	.2489	.6093
CAE++	.2838	.2193	.0882	.2715	.2452	.1901	.2648	.1048	.1125
Slot Attention	.2872	.2220	.0801	.1739	.2192	.1071	.1930	.0876	.0680
CtCAE	.2357	.2148	.0964	.2681	.2420	.2034	.3079	.1158	.1609
CAE	.1091	.2008	.0634	.1526	.2305	.1192	.1200	.1038	.0630
Best baseline, per cell	.6326	.4267	.3853	.7202	.7540	.5913	.5505	.6834	.6348
Our margin	+.2082	+.3261	+.2534	+.1366	+.0410	+.2257	+.1461	+.1356	+.1277

Table 1: All models trained on the same 10,000 CLEVR images for 10 epochs and evaluated on 320 held-out images per dataset by the same function; Tetrominoes and Multi-dSprites are zero-shot. Metrics are foreground ARI, foreground IoU and matched-object IoU on the 64×64 patch grid. Every model, ours included, has the minimum-region-size rule of Section 3.1 at f = 0.01. The penultimate row takes the best baseline in each cell separately, which no single baseline attains. Single training seed throughout. The baselines' cluster count and background mapping are selected on each target dataset's own validation split, ours on CLEVR validation only; Appendix E states each model's conditions.

the oscillatory gate g at correlation 0.9997, and the identical readout run on the gates reproduces every mask exactly, on all 40 images and to every digit. The necessary timing is therefore the gate's, transmitted by the spiking layer without loss; on this evidence no separate grouping information can be attributed to the firing threshold.

Predicted foreground. Against true fractions of 0.2115 on CLEVR and 0.0943 on Multi-dSprites, our readout predicts 0.1988 and 0.0827 against the baselines' 0.41 to 0.86 on CLEVR: the only method that does not over-claim.

Budget and released weights. Slot Attention's released weights, at roughly $320\times$ our exposure budget, score lower in all nine cells, by 0.377 to 0.720. Two of the six baselines reach their own published scores at 1/210 and 1/64 of their schedules, so those rows are not artefacts of the shortened schedule; for the other four the comparison is a data-efficiency claim. Appendix F gives both in full with the rollout-length and data-scaling studies, Appendix G the per-model tables.

3.3 Limitations

Object count. Our readout groups patches better than it counts objects: with the rule it predicts 5.15 objects per image on CLEVR against a true 6.36 (exact on 25.0%), 2.64 against 2.49 on Multi-dSprites (53.1%) and 5.28 against exactly 3 on Tetrominoes (9.4%). Without the rule it predicts 8.04, 6.62 and 21.07, so most of the error was single-patch fragments rather than over-segmentation. Choosing f on each target's own validation removes most of the rest — f = 0.10 gives 3.02 objects on Tetrominoes, 86.9% exact — but that reads target labels, so the transferred value is the headline.

Two cautions. Tetrominoes contains exactly three objects in every image, so always answering three is right without grouping anything: the fixed-k baselines all score 100% there, CtCAE included at fg-ari 0.0756. The count is informative only where the true count varies, and on Multi-dSprites it does — those baselines score 0% exact against our 53.1%. The residual error also follows from the objective: we train without a reconstruction term, which is what penalises splitting an object into parts. Every baseline in Table 1 optimises one; ours does not.

Spiking nonlinearity. The firing indicator is saturated over the readout window (Section 3.2), so our evidence supports oscillatory timing and does not isolate a contribution from the threshold nonlinearity. Whether a sparser firing regime would add grouping information is a question about the training objective, which places no constraint on the rate.

Seeds. Every number here is a single training seed. Reseeding one baseline moved fg-ari by up to 0.134 on CLEVR; all margins in Table 1 exceed that, but this is evidence about baseline variance, not an error bar on our own model.

Protocol. The held-out split is reserved from the training corpus rather than an independent benchmark test set, and figures from other papers appear only as each model's comparison with itself (Appendix F), never as entries in Table 1.

4. Conclusion

An oscillatory spiking network can group image patches into object masks without mask supervision during training or a prespecified object count at inference. Under a shared evaluation protocol it reached 0.841 FG-ARI on CLEVR against 0.633 for the best evaluated baseline, and scored higher on Tetrominoes and Multi-dSprites without additional training, at a single training seed. Ablations place the grouping signal in the cross-patch alignment of the oscillatory gate rather than in the firing threshold, so the result supports relative oscillatory timing as a cue for object discovery. The rollout is costly, counting is weaker than grouping, and whether the gated traces also support object tracking is left to future work.

References

Dan, T.; and Wu, G. 2026. From Cortical Synchronous Rhythm to Brain Inspired Learning Mechanism: An Oscilla-

tory Spiking Neural Network with Time-Delayed Coordination. arXiv:2605.01656.

Fries, P. 2005. A mechanism for cognitive dynamics: neuronal communication through neuronal coherence. *Trends in Cognitive Sciences*, 9(10): 474–480.

Gray, C. M.; König, P.; Engel, A. K.; and Singer, W. 1989. Oscillatory responses in cat visual cortex exhibit intercolumnar synchronization which reflects global stimulus properties. *Nature*, 338: 334–337.

Locatello, F.; Weissenborn, D.; Unterthiner, T.; Mahendran, A.; Heigold, G.; Uszkoreit, J.; Dosovitskiy, A.; and Kipf, T. 2020. Object-Centric Learning with Slot Attention. In *Advances in Neural Information Processing Systems*.

Mainen, Z. F.; and Sejnowski, T. J. 1995. Reliability of spike timing in neocortical neurons. *Science*, 268(5216): 1503–1506.

Miyato, T.; Löwe, S.; Geiger, A.; and Welling, M. 2025. Artificial Kuramoto Oscillatory Neurons. In *International Conference on Learning Representations*.

Appendix

A Sensory Representation and Graph Details

The input is a 128×128 RGB image scaled by $\frac{1}{255}$. A single 3×3 convolution with eight trainable filters, unit stride, and no padding produces $126 \times 126 \times 8$ feature maps. There is no post-convolution activation. All eight channels share a scalar mean μ and standard deviation σ, estimated once from the initial encoder on training RGB images. We use $\sigma = \sqrt{\max(\mathrm{Var}(F), 10^{-8})}$, where F pools feature values over training images, channels, and spatial positions. We standardize with these fixed statistics and clip the result to $[-3, 3]$. Adaptive average pooling produces a 64×64 grid, yielding $N = 4096$ patch vectors with eight features each.

The bias-free trainable projection W gives the embedding

$$\boldsymbol{z}_i = \frac{W\gamma_i}{\max(\|W\gamma_i\|_2, \varepsilon)}. \tag{A1}$$

For grid coordinates p_i, the scaled spatial distance and local traversal cost are

$$\begin{aligned} d_{\mathrm{sp}}(i,j) &= \frac{\|p_i - p_j\|_2}{4}, \\ c_{ij} &= d_{\mathrm{sp}}(i,j)[1 + 8(1 - \boldsymbol{z}_i^\top \boldsymbol{z}_j)^2]. \end{aligned} \tag{A2}$$

Set $d_{ij}^{(0)} = c_{ij}$ when $d_{\mathrm{sp}}(i,j) \le 1.5$ and 16 otherwise. With $\tau_{\mathrm{geo}} = 0.5$, three updates ($m = 0, 1, 2$) give

$$\begin{aligned} u_{ij}^{(m)} &= -\tau_{\mathrm{geo}} \log \sum_k \exp\left(-\frac{d_{ik}^{(m)} + d_{kj}^{(m)}}{\tau_{\mathrm{geo}}}\right) \\ &\quad + \tau_{\mathrm{geo}} \log 4, \\ d_{ij}^{(m+1)} &= \min(d_{ij}^{(m)}, u_{ij}^{(m)}), \\ d(i,j) &= \min(d_{ij}^{(3)}, 16). \end{aligned} \tag{A3}$$

With learned scalars $\ell_\tau, \ell_G, \ell_r$, the graph parameters are

$$\begin{aligned} \tau &= \max(\exp(\ell_\tau), 0.001), \qquad G = \exp(\ell_G), \\ r &= \mathrm{softplus}(\ell_r) = \log(1 + \exp(\ell_r)). \end{aligned} \tag{A4}$$

Each patch selects its 512 highest-scoring connections before normalization. Symmetrization can increase its degree beyond 512.

The trainable projection, W_{drive}, maps eight features to four components without bias. Its output is standardized over all $4N$ patch-component entries of the current image with a standard deviation floor of 10^{-6}. The trainable scalar, W_ϕ, scales this input before tanh bounds the drive between $-\pi$ and π. The drive and graph remain fixed during a rollout. The input phase is computed as

$$\begin{aligned} \boldsymbol{u}_i &= W_{\mathrm{drive}}\gamma_i, \\ \boldsymbol{\phi}_i = \pi \tanh(W_\phi \bar{\boldsymbol{u}}_i), &\qquad \boldsymbol{\theta}_i(0) = \boldsymbol{\phi}_i. \end{aligned} \tag{A5}$$

B Recurrent Dynamics

B1 Coupling Correction and Phase Lag The corrected graph stabilizes inverse connection costs and gives zero graph entries a small residual weight:

$$\begin{aligned} \bar{A} &= \mathrm{ReLU}\left(\frac{A + A^\top}{2}\right) + 10^{-6}, \\ C_{ij} &= \frac{1}{\bar{A}_{ij}}. \end{aligned} \tag{A6}$$

$$\begin{aligned} \bar{C}_{ij} &= \frac{C_{ij} - \min C}{\max C - \min C + 10^{-6}}, \\ \alpha_{ij} &= \tanh(R_{ij} - R_{ji})\bar{C}_{ij}. \end{aligned} \tag{A7}$$

The inverse connection cost, C_{ij}, is normalized over all patch pairs to obtain $\bar{C}_{ij}$. The trainable directional matrix, R, determines the direction of the phase lag, α_{ij}, through its antisymmetric difference. The inverse cost sets the lag scale. This permits coordinated phases with nonzero offsets. The lag is shared by all four components.

The natural frequency, $\boldsymbol{\omega}_i$, controls intrinsic angular drift. The drive adjusts frequency through $f\boldsymbol{\phi}_i$ and provides sine forcing toward $\boldsymbol{\phi}_i$ when the drive strength, $\boldsymbol{\kappa}_i$, is positive. Coupling, drive, and drift act together; phase is not required to converge to the input phase. The coupling scale, K, equals N, and the time step, Δt, is 0.1.

B2 Rhythmic Gating and Branch Integration We set the phase delay to $d = 2$ simulation steps. The scalar gate, $g_i(t)$, is shared across the patch's four components and has no trainable parameters.

The mean patch phase averages the four component phases:

$$\bar{\theta}_i(t) = \frac{1}{4}\sum_{a=1}^{4} \theta_{ia}(t). \tag{A8}$$

Each branch state vector has four entries. The retention vector, $\boldsymbol{\beta}_i$, is obtained by applying sigmoid to the trainable retention vector, $\boldsymbol{k}_i$. The provisional branch state is

$$\begin{aligned} \widehat{\boldsymbol{h}}_{ia}(t) &= \boldsymbol{\beta}_i \odot \boldsymbol{h}_{ia}(t-1) \\ &\quad + (1 - \boldsymbol{\beta}_i) \odot \boldsymbol{v}_{ia}(t). \end{aligned} \tag{A9}$$

Retention is shared across oscillator components of the same patch, while their branch states remain separate. Modulating the full increment also controls leakage and spike recurrence,

which would continue if only the current input were gated. Summing the four entries combines branch responses without averaging oscillator components.

B3 Membrane Update and Spike Emission The provisional membrane voltage is

$$\widehat{V}_{ia}(t) = \lambda_i V_{ia}(t-1) + (1-\lambda_i)H_{ia}(t) - \vartheta_a s_{ia}(t-1). \tag{A10}$$

The membrane retention, λ_i, is the sigmoid of the trainable patch parameter, ℓ_i. The firing threshold, ϑ_a, is 0.06 times the exponential of the trainable component parameter, q_a. Membrane resistance is fixed to one. The subtractive reset is proportional to the previous emitted spike and does not force voltage to zero.

When the gate is exactly zero, the previous voltage is retained. Otherwise, the gated update is applied before thresholding. The binary firing event, $e_{ia}(t)$, is multiplied by the gate to produce the emitted spike, $s_{ia}(t)$, which enters the next branch input and membrane reset. Dendritic and membrane states and the previous spike start at zero. The rollout uses 1024 steps.

C Temporal Affinity and Object Masks

We retain steps $\mathcal{T} = \{512, \dots, 1023\}$ and index them by $m = 1, \dots, 512$. Centering and L2 normalization give

$$\mu_{ia} = \frac{1}{512}\sum_{t\in\mathcal{T}} s_{ia}(t), \qquad r_{ia,m} = \frac{s_{ia}(511+m) - \mu_{ia}}{\max\left(\sqrt{\sum_{t\in\mathcal{T}}(s_{ia}(t)-\mu_{ia})^2}, 10^{-8}\right)}. \tag{A11}$$

A constant trace becomes zero. The component correlation and patch affinity are

$$\rho_{ija} = \operatorname{clip}\left(\sum_{m=1}^{512} r_{ia,m}\, r_{ja,m}, -1, 1\right), \tag{A12}$$

$$Q_{ij} = \prod_{a=1}^{4} \max(\rho_{ija}, 0). \tag{A13}$$

Centering removes the baseline and normalization reduces sensitivity to absolute scale. The product requires positive temporal agreement in every component; one nonpositive correlation makes the affinity zero. The affinity is not a calibrated same-object probability.

Connected components of $Q \geq 0.5$ determine the initial support without a spatial adjacency constraint. Components smaller than 8 patches are discarded, a floor selected on CLEVR validation, and the largest remaining component is treated as background. Every patch enters this step as a candidate; no activity or firing-rate threshold is applied, so the foreground is defined entirely by the synchrony structure. Removed patches are not recovered. The retained support is reset to singleton patches before merging.

The stabilized affinity is

$$Q' = \operatorname{clip}\left(\frac{Q+Q^\top}{2}, 10^{-6}, 1-10^{-6}\right). \tag{A14}$$

The merge evidence, E, is symmetrized and its diagonal set to zero. The log-odds term gives positive evidence above 0.5 and negative evidence below 0.5. Graph weights change its magnitude without changing its sign. Merge scores sum all cross-group pairs. Only groups adjacent on the four-neighbor grid are candidates. The largest positive candidate is merged until none remains.

The hard label, y_i, identifies the final foreground group or equals zero for excluded patches. The binary object mask, M_k, selects the patches assigned to group k on the 64×64 grid. The 8-patch floor is not reapplied after merging. If no foreground survives, the output is all background.

D Training Details

D1 Initialization and optimization We train the Native64 encoder and recurrent core jointly from random initialization without loading a pretrained checkpoint. The shared dendritic bias starts at zero. Training uses RGB images without segmentation masks, object counts, or reconstruction targets.

We compute the objective separately for each image and average the resulting losses across images. After accumulating gradients, we clip the global ℓ_2 norm across trainable parameters to 1 and apply Adam. The encoder and the other active parameter groups use separate fixed learning rates. The legacy geodesic contrast parameter is excluded from optimization because the forward operator uses a fixed contrast.

D2 Rollout and gradient window We simulate 1,024 steps per image. Both losses use steps 512–1023 (zero-based), but recurrent backpropagation covers only steps 960–1023. The first 960 steps run without gradient tracking, and the recurrent states are detached at the boundary. The earlier steps in the loss window still contribute to the statistics. The image graph, sensory drive, and factorized coupling are computed with gradients enabled and reused in the final 64 steps, so the loss can reach the encoder through the graph and drive. The phase loss propagates through the Kuramoto dynamics, while the spike loss also trains the dendritic and membrane parameters through the emitted spike traces. A multi-Gaussian surrogate derivative supports backpropagation through the binary spike threshold. Connected-component filtering and greedy merging are used for inference and do not enter the training objective.

D3 Phase and spike statistics The phase loss uses a phase-locking value (PLV) matrix P. Using the arithmetic mean phase $\bar{\theta}_i(t)$ from Eq. (A8), define $\delta_{ij}(t) = \bar{\theta}_i(t) - \bar{\theta}_j(t)$ and let $\langle\cdot\rangle_t$ denote the mean over steps 512–1023. Then

$$R^{\mathrm{plv}}_{ij} = \langle\cos\delta_{ij}(t)\rangle_t, \qquad I^{\mathrm{plv}}_{ij} = \langle\sin\delta_{ij}(t)\rangle_t, \qquad P_{ij} = \operatorname{clip}\left(\sqrt{(R^{\mathrm{plv}}_{ij})^2 + (I^{\mathrm{plv}}_{ij})^2 + 10^{-12}}, 0, 1\right). \tag{A15}$$

A high PLV reflects a stable phase difference, which need not be zero. The spike loss uses the affinity matrix Q from Appendix C, computed from the emitted, gated spikes over the same window. Unlike P, Q multiplies the nonnegative correlations of four separate spike traces.

D4 Loss function For $X \in \{P, Q\}$, let $\langle\cdot\rangle_{\neq}$ be the mean over all $N(N-1)$ ordered off-diagonal pairs. We use the same criterion for both matrices:

$$\begin{aligned} \mathcal{C}(X) &= 6L_{\text{bim}}(X) + 10L_{\text{bal}}(X) \\ &\quad + 0.5L_{\text{coh}}(X) + L_{\text{col}}(X), \\ L &= \mathcal{C}(P) + 5\mathcal{C}(Q). \end{aligned} \tag{A16}$$

Let $\mu_X = \langle X_{ij}\rangle_{\neq}$ and $v_X = \langle (X_{ij} - \mu_X)^2\rangle_{\neq}$. The bimodality, balance, and collapse terms are

$$\begin{aligned} L_{\text{bim}}(X) &= \langle X_{ij}(1 - X_{ij})\rangle_{\neq}, \\ L_{\text{bal}}(X) &= (\mu_X - 0.867)^2, \\ L_{\text{col}}(X) &= -\log(v_X + 10^{-4}). \end{aligned} \tag{A17}$$

The first favors affinities near zero or one; the second targets a fixed mean affinity of 0.867; the third penalizes low off-diagonal variance. The target is a pairwise affinity prior, not a target for object count or foreground area. The logarithmic penalty is finite at zero variance.

For spatial coherence, we reshape the off-diagonal row means $a_i = (N-1)^{-1}\sum_{j\neq i} X_{ij}$ into a 64×64 field $a_{r,c}$ and use

$$\begin{aligned} L_{\text{coh}}(X) = {} & \text{mean}_{r,c}\,|a_{r+1,c} - a_{r,c}| \\ & + \text{mean}_{r,c}\,|a_{r,c+1} - a_{r,c}|. \end{aligned} \tag{A18}$$

Each mean includes only valid adjacent pairs in its direction. This term smooths each patch's mean affinity to the other patches; it does not operate on the final masks.

E Evaluation Conditions

E1 What each metric can and cannot see All three metrics take two aligned integer label maps on the 64×64 patch grid, with background as label 0 on both sides. They differ in which patches they look at, and the difference decides which errors are charged.

Foreground ARI restricts both label maps to the patches whose *ground truth* is foreground, and computes the adjusted Rand index of the two partitions of that set. Predicted background is kept as an ordinary cluster inside it, so predicting background on a true object is charged; but patches whose ground truth is background are removed before scoring, so *labelling background as object is invisible to this metric*. An image with fewer than two true foreground patches is undefined and excluded; a degenerate pair of identical partitions scores 1.

Foreground IoU merges every non-background identity on each side into one set and reports $|\hat{F} \cap F|/|\hat{F} \cup F|$ over patches. False foreground enters the union and nothing else, so this is the metric that charges for claiming background as object — the error FG-ARI cannot see. Both sets empty scores 1; exactly one empty scores 0. This is why Table 1 is read with the two together: a method that labels most of the image as object can hold a respectable FG-ARI while its foreground IoU collapses, which is what the F-IOU column shows for the baselines that predict 0.41 to 0.86 foreground against a truth of 0.2115.

Matched-object IoU solves a one-to-one Hungarian assignment between predicted and true objects maximising total IoU, sums the matched IoUs and divides by the number of *true* objects, so an unmatched true object contributes zero. Background is not matched. False foreground enlarges the unions of the objects it touches, but *extra predicted objects carry no direct penalty* beyond going unmatched. Over-segmentation is therefore not charged by this metric, which is the reason object count is reported separately in Section 3.3 rather than treated as covered by it. No true objects gives an undefined score; predictions empty against a non-empty truth gives 0.

Undefined images are excluded per metric and the remainder averaged, so the three metrics can rest on slightly different image counts; the counts are recorded with every result. No weighted combination of the three is formed anywhere.

E2 Which split fixed each free choice Every method has choices that are not learned by its training objective, and a transfer claim is only as strong as the data used to fix them. *Not trained on the target* and *not tuned on the target* are different statements, and they do not hold equally for every row of Table 1.

Ours. One free scalar: the minimum-region-size fraction f. It is fixed at 0.01 on the *CLEVR* validation split and applied unchanged to Tetrominoes and Multi-dSprites. No Tetrominoes or Multi-dSprites data, labelled or unlabelled, enters any choice we make. Everything else in the readout — the $Q \geq 0.5$ link threshold, the 8-patch component floor (itself selected on CLEVR validation), the largest-component background heuristic, the rollout length and the readout window — is fixed before these datasets are touched and is identical across all three.

Baselines. Two free choices, and *both are fixed on the target dataset's own validation split*: the number of clusters k and the rule mapping one cluster to background. The background rule is selected from four candidates (global-largest, image-largest, border-largest, border-union) by validation FG-ARI with foreground IoU as tie-break, separately for each model and each dataset. For the CAE family k is likewise selected per dataset from $\{4, 7, 11\}$, and the selection does not land on the same value everywhere: CAE++ uses $k = 11$ on CLEVR and Tetrominoes but $k = 7$ on Multi-dSprites, and CAE uses 11, 11 and 4. For AKOrN, ItrSA and Slot Attention the slot count is the architecture's own and is left at the authors' value, but the background rule is still chosen on the target's validation split. The selection used 320 validation images on CLEVR, and 1,000 on Tetrominoes and Multi-dSprites for the AKOrN family, 320 for the CAE family.

The asymmetry runs in the baselines' favour, not ours: on the two transfer datasets they are allowed to see target labels to set k and the background mapping, and we are not. We state it here rather than leaving "zero-shot" to carry both meanings at once. Removing the oracle that the reference implementations of CAE, CAE++, CtCAE and SynCx use — setting k_i from the per-image ground-truth object count — is a separate and stricter correction, described in Section 3.1; a per-dataset constant chosen on validation is what replaces it.

The minimum-region-size rule is offered to every model

T	FG-ARI	FG-IOU	OBJ-IOU	Δ FG-ARI
1024	.8421	.7545	.6410	—
512	.8402	**.7594**	.6340	$-.0019$
256	.8159	.7358	.6084	$-.0262$
128	.5796	.4988	.3985	$-.2625$

Table 2: CLEVR, model fixed, readout window $T/2$. Halving the rollout is free; a quarter costs 0.02–0.03; an eighth fails. This table and the scaling figures below are reported *without* the minimum-region-size rule, because both are within-model comparisons at a fixed readout and the rule would shift every row by the same small amount.

on the same terms, and it is not a private advantage: applied at $f = 0.01$ it leaves every baseline row of Table 1 unchanged to three decimals except AKOrN (conv) on CLEVR foreground IoU, which improves from .4219 to .4267; the best-baseline row uses the better of the two for every baseline cell we measured.

No hyperparameter, threshold or rule of any model is selected on the 320 reserved images that Table 1 reports. Those images are read once, after every choice is frozen.

F Additional Experiments

F1 Budget and released weights Two conditions on Table 1 deserve measurements rather than argument: that the baselines are evaluated away from their published setting, and that the matched budget limits them.

Released weights. Only Slot Attention ships trained weights; the AKOrN, CAE and SynCx repositories release data but no checkpoints. Evaluated directly, at roughly $320\times$ our exposure budget, its released model scores .3747/.2219/.1165 on CLEVR, .4742/.2381/.2355 on Tetrominoes and .3205/.0980/.1204 on Multi-dSprites. Our model scores higher in all nine cells, by 0.377 to 0.720, so the comparison does not rest on our retraining of it. The extra data also helps it far more *off* its training distribution than on it — worth $+.0875$ FG-ARI on CLEVR but $+.3003$ on Tetrominoes and $+.1275$ on Multi-dSprites — so a data-efficiency conclusion drawn from an in-domain column alone would not transfer.

Budget. Comparing each baseline against its own published number, by its own evaluation code, isolates the effect of the shortened schedule. Two of the six are *not* budget-limited: AKOrN (conv) reaches 60.5 at $1/210$ of its schedule against a published 59.0 ± 4.3, and CAE reaches 10.3 at $1/64$ against a published 4.0. For those two the margin in Table 1 is not an artefact of a short schedule. The other four are limited — AKOrN (attn) 60.4 against 91.0 ± 0.5, ItrSA 58.1 against 82.5 ± 1.7, CAE++ 16.7 against 27.0, and CtCAE 12.0 against 54.0 — and against those the result should be read as a data-efficiency claim. We state both readings rather than only the favourable one. Full figures are in Appendix G.

F2 Rollout length and data scaling The readout integrates spiking activity over T simulation steps and reads the second half of the window. Table 2 varies T with the model fixed.

Halving the rollout costs 0.0019 FG-ARI — below any difference we can resolve at one seed — and *improves* foreground IoU. Performance degrades gently to $T = 256$ and collapses at $T = 128$, where the model also under-segments, predicting a foreground fraction of 0.1148 against a truth of 0.2115. We therefore use $T = 512$ as the default operating point and $T = 1024$ for the reported numbers. Shorter windows also fragment more: mean predicted regions per image rise from 8.0 at $T = 1024$ to 13.3 at $T = 256$, against 6.36 true objects.

Scaling with training data. Continuing training on additional unseen CLEVR images, in steps of 1,000 from 10,000 to 18,000, separates the three metrics. FG-ARI runs .8421, .8426, .8449, .8460, .8469, .8459, .8456, .8470, .8461: it gains .0028 over the first two steps and then stays inside a band of .0021 for the remaining six, with no trend. Foreground IoU runs .7545 to .7755 and matched-object IoU .6410 to .6579, both still rising at 18,000 and neither turning. More in-domain data therefore stops improving *how* patches are grouped at around 12,000 images while it is still improving *where the boundaries fall*. Mean predicted regions hold at 7.7–8.0 throughout, against 6.36 true objects, so the extra data sharpens the partition without changing how much it splits. The FG-ARI band is one seed wide and we read it as flat rather than converged; the two IoU curves have not been run far enough to find their own ceiling.

In-domain data is not the most useful data. The same 5,000 additional images buy more than twice as much when they come from a different dataset. Continuing on CLEVR raises CLEVR FG-ARI by $+.0038$; continuing on Tetrominoes instead raises it by $+.0084$, a $2.2\times$ larger gain at an identical budget. Together with the transfer result in Table 1, this argues that the model is not fitting a domain but learning a grouping rule that the extra renders refine.

F3 Oscillatory timing, measured The three measurements summarised in Section 3.2, in full. All run on the first 40 images of the CLEVR reserved split, on the same rollouts, so they are directly comparable. They are reported in the order they answer the hypothesis: which pattern is observed, whether its temporal alignment is necessary, and which signal carries it.

Validity of the ablation. The affinity Q is recomputed here from `component_spikes` rather than taken from the model, so that a modified spike trace can be fed through the same arithmetic. On the unshifted control the recomputed Q agrees with the model's own to 6.7×10^{-6} maximum absolute difference over all 4096^2 entries, so the ablations differ from the control in exactly one thing.

Separability. Ground-truth labels are used only to sort patch pairs into three groups and never to set a threshold or a count. Averaged over the 40 images, the affinity is 0.891 ± 0.111 for pairs inside one object, 0.047 ± 0.048 for pairs in different objects, and 0.099 ± 0.106 for object–background pairs. The within-object value is nineteen times the across-object value, and the across-object value sits near the floor of the measure.

	FG-ARI	FG-IOU	OBJ-IOU	regions
control	.8648	.8028	.6925	7.8
per-patch shift	.0735	.7358	.0325	562.3
gates, not spikes	.8648	.8028	.6925	7.8

Table 3: First 40 CLEVR reserved images. Destroying cross-patch timing collapses the partition (FG-ARI, OBJ-IOU) while barely moving foreground IoU. The whole readout, foreground decision included, is recomputed from the shifted traces; foreground IoU survives because it compares only the *union* of predicted non-background patches against the true foreground and is invariant to how that union is partitioned, and shattering one region into 562 leaves the union nearly unchanged. Substituting the phase gates for the spikes changes nothing at all.

Per-patch circular shift. Each patch's four component traces are rolled by one offset drawn uniformly over the readout window, independently per patch. Rate, waveform and within-patch structure survive; only the alignment between patches is destroyed.

The readout's *rules* are unchanged — the image-conditioned graph, the $Q \geq 0.5$ link threshold, the 8-patch component floor, the largest-component background heuristic and the greedy merge — but every quantity they operate on is recomputed from the shifted traces. In particular the foreground is *not* held fixed and is *not* rate-based: the readout passes every patch to the grouping step as a candidate and takes the largest surviving synchrony component as background, so the foreground decision is itself a function of the shifted affinity. The ablation therefore re-runs the whole readout and differs from the control in one input only.

Gates in place of spikes. The third row replaces `component_spikes` with `component_gates` and changes nothing else. Every mask is reproduced exactly, on all 40 images and to every digit of all three metrics. The reason is visible in the traces: with $s_{ia} = g_i e_{ia}$, the firing indicator e equals 1 in 99.992% of settled steps, the correlation between s and g is 0.9997, and the 0.008% of steps where the neuron is silent never change a thresholded decision. The timing that the shift ablation shows to be necessary is therefore the timing of the oscillatory gate. The spiking layer carries it without loss, but the readout cannot tell the two apart, and no claim should be made for the threshold nonlinearity on this evidence.

G Supplementary Tables

G1 Released weights These figures are given in prose in the main paper.

G2 Training budget per baseline

G3 Scaling with training data

	CLEVR	Tetrominoes	Multi-dSprites
Slot Attention, matched budget	.2872 / .2220 / .0801	.1739 / .2192 / .1071	.1930 / .0876 / .0680
Slot Attention, released weights	.3747 / .2219 / .1165	.4742 / .2381 / .2355	.3205 / .0980 / .1204
Ours, matched budget	**.8421 / .7545 / .6410**	**.8580 / .7885 / .8173**	**.6974 / .8177 / .7628**

Table 4: Released weights ($\approx$32M exposures) against the matched-budget retraining ($100{,}000$ exposures) and against our model. Triples are ARI / F-IOU / O-IOU. All three rows are reported *without* the minimum-region-size rule, so the released and retrained Slot Attention rows are compared on identical terms; Table 1 of the main paper applies the rule to every model and our row there reads $.8408/.7528/.6387$, $.8568/.7950/.8170$ and $.6966/.8190/.7625$.

Model	Budget	Ours	Published
AKOrN (conv)	$1/210$	**60.5**	59.0 ± 4.3
CAE	$1/64$	**10.3**	4.0
AKOrN (attn)	$1/210$	60.4	91.0 ± 0.5
ItrSA	$1/210$	58.1	82.5 ± 1.7
CAE++	$1/64$	16.7	27.0
CtCAE	$1/64$	12.0	54.0

Table 5: Each baseline at our budget against its own published figure, by its own metric and evaluation code. *Budget* is the fraction of the published training schedule our matched-exposure protocol gives it. The top block is *not* budget-limited: AKOrN (conv) exceeds its published CLEVR number at $1/210$ of its schedule and CAE exceeds its own at $1/64$. The bottom block is.

Images	FG-ARI	FG-IOU	OBJ-IOU	Δ FG-ARI
10,000	.8421	.7545	.6410	—
11,000	.8426	.7539	.6427	$+.0005$
12,000	**.8449**	**.7607**	**.6461**	$+.0028$

Table 6: *Images* is the cumulative count of unique training images. Each row adds $1{,}000$ previously unseen CLEVR images for 10 epochs, resuming from the previous row. CLEVR evaluation.

Symbol	Shape	Meaning and status
γ	$N \times 8$	Computed patch activations
W	16×8	Trainable graph projection
$\boldsymbol{z}_i$	16	Computed normalized patch embedding
S, d, c	$N \times N$ each	Computed scores, geodesic distances, and local costs
τ, r, G	Scalar each	Positive values derived from trainable scalars
A, B	$N \times N$ each	Computed connection matrices
W_{drive}, W_ϕ	4×8; scalar	Trainable input projection and phase gain
$\boldsymbol{u}, \bar{\boldsymbol{u}}, \boldsymbol{\phi}$	$N \times 4$ each	Computed input representations

Table 7: Variable meanings, shapes, and parameter status for the sensory representation.

Symbol	Shape	Meaning and status
$\boldsymbol{\theta}(t), \boldsymbol{\omega}, \boldsymbol{\kappa}$	$N \times 4$ each	Dynamic phase; trainable natural frequencies and drive strengths
f	Scalar	Trainable frequency gain
$K, \Delta t$	Scalar each	Fixed coupling scale and time step
R	$N \times N$	Trainable directional matrix
$\bar{A}, C, \bar{C}, \alpha$	$N \times N$ each	Computed corrected graph, inverse costs, normalized costs, and lags
$\bar{\theta}(t), g(t)$	N each	Computed mean phases and gates
$\boldsymbol{x}(t)$	$N \times 4$	Computed gated features
$U, \boldsymbol{b}$	4×2; 4	Trainable branch projection and bias
$\boldsymbol{v}_{ia}(t), \boldsymbol{h}_{ia}(t), \widehat{\boldsymbol{h}}_{ia}(t)$	4 each	Computed branch input and states
$\boldsymbol{k}, \boldsymbol{\beta}$	$N \times 4$ each	Trainable retention parameters and their sigmoid transforms
η, ξ	4 each	Trainable dendritic and membrane modulation weights
$H(t), V(t), \widehat{V}(t), e(t), s(t)$	$N \times 4$ each	Computed membrane input, voltages, events, and spikes
ℓ, λ	N each	Trainable retention parameters and their sigmoid transforms
q, ϑ	4 each	Trainable threshold parameters and their derived thresholds

Table 8: Variable meanings, shapes, and parameter status for the recurrent core.

Symbol	Shape	Meaning and status
$\boldsymbol{r}_{ia}$	512	Computed normalized temporal trace
ρ	$N \times N \times 4$	Computed component correlations
Q, Q', E	$N \times N$ each	Computed affinity, stabilized affinity, and signed evidence
Δ	Scalar per pair	Computed merge score
y	N or 64×64	Computed hard labels
M_k	N or 64×64	Computed binary mask for object group k

Table 9: Variable meanings, shapes, and parameter status for the mask readout.